

Serving LLMs in Production

Delivering Scalable, Reliable, Low-Latency AI

A practical guide for ML engineers and SREs building enterprise-grade LLM infrastructure



Why Serving Is Challenging

Size Constraints

Models are enormous, creating significant loading time and GPU memory strain

Performance Demands

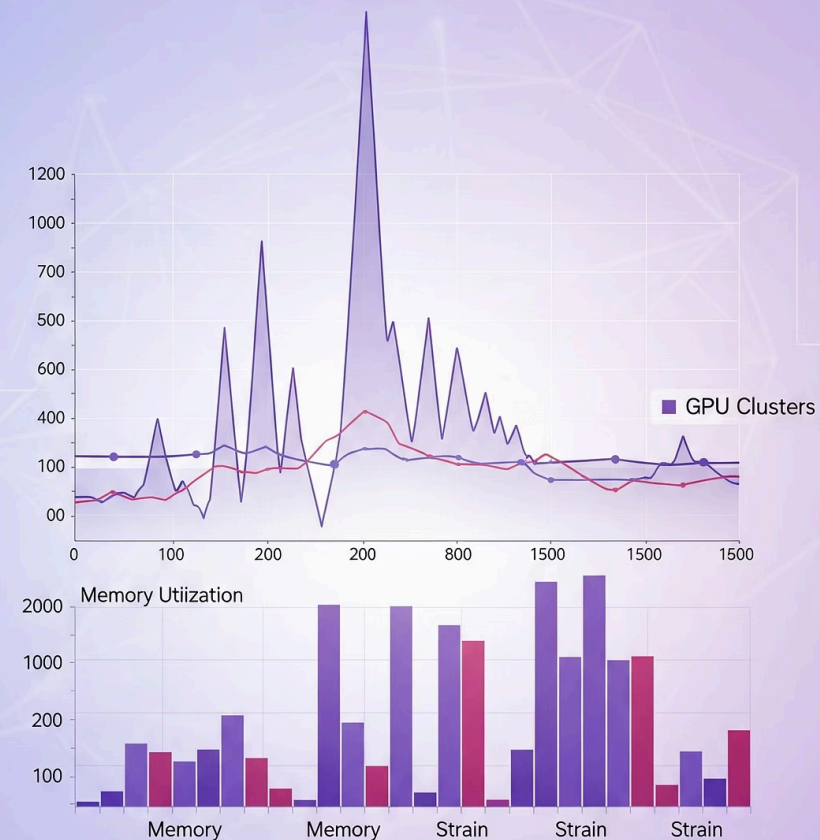
Users expect **sub-second responses** regardless of complexity

Traffic Unpredictability

Spiky, unpredictable query patterns require elastic infrastructure

Resource Efficiency

Need **multi-tenant, cost-aware infrastructure** to manage GPU expenses



Core Serving Architecture

A robust LLM serving stack requires these essential components working in concert:



Frontend API

Receives and validates user queries, handles authentication and rate limiting



Inference Engine

Executes model on accelerated hardware (GPU/TPU), manages execution graphs



Caching Layer

Stores prompt results, KV cache states, and embedding vectors to reduce redundant computation



Orchestration

Handles autoscaling, load balancing, and intelligent request routing



Observability

Collects logs, metrics, and distributed traces for performance analysis

Model Deployment Options

Single-Node Serving

Simple deployment, limited scale

- Easy setup with Hugging Face or FastAPI
- Limited by single GPU memory capacity
- Suitable for smaller models (< 13B parameters)

Distributed Serving

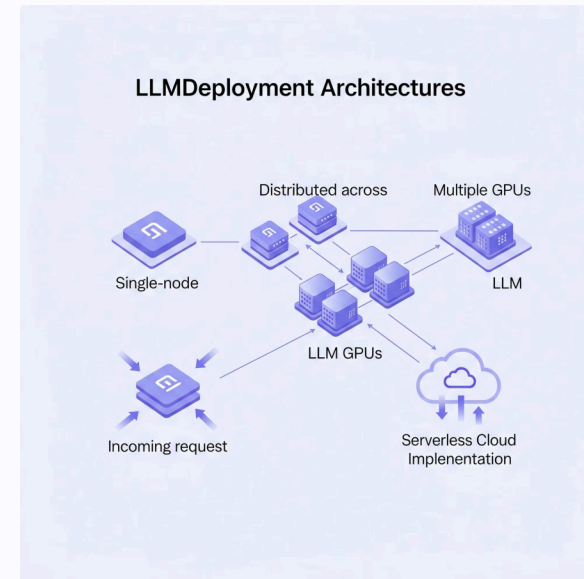
Sharded models across GPUs/nodes

- Tensor parallelism across multiple GPUs
- Supports larger models (70B+)
- Higher operational complexity

Serverless APIs

On-demand, cost-efficient, higher latency

- Pay-per-token pricing model
- Cold start penalties
- No infrastructure management



Inference Runtimes: TensorRT, vLLM, DeepSpeed-Inference offer specialized optimizations for different deployment models

Performance Optimization



"Every millisecond of inference latency directly impacts user experience in conversational AI."



KV-Cache Reuse

Store key-value pairs from previous tokens to cut latency in long contexts by up to 50%



Request Batching

Group similar requests for higher throughput, balancing latency trade-offs



Quantization & Pruning

Reduce model precision from FP32 to INT8/INT4 for smaller memory footprint



Memory Pinning

Keep frequently used models in GPU memory to avoid costly reload delays



Reliability & Scaling

Dynamic Scaling

Autoscale GPU pods based on demand using Kubernetes HPA or Ray Serve cluster

- Scale based on queue depth, not just CPU/memory
- Preemptive scaling for predicted traffic spikes

Safe Deployment

Canary rollouts for new models & versions with automatic rollback capabilities

- A/B testing for model performance evaluation
- Shadow traffic for risk-free validation

1

2

3

4

Global Reach

Multi-region deployment with intelligent routing for optimal user experience

- Geographically distributed inference endpoints
- Data sovereignty compliance for regulated industries

Resilience

Implement fallback strategies: degrade to smaller/faster models under load

- Circuit breakers to prevent cascading failures
- Graceful degradation pathways

Observability & Monitoring

Performance Metrics

Track **latency (p50, p95, p99)**, **throughput**, **error rates** at both API and model levels

Resource Utilization

Monitor GPU utilization, memory fragmentation, and CUDA kernel performance

Distributed Tracing

Implement end-to-end request tracing with OpenTelemetry to identify bottlenecks

Proactive Alerting

Set up alerts for model drift, inference anomalies, and service degradation



Comprehensive monitoring enables rapid diagnosis and resolution of production issues



Security & Governance

Securing LLM infrastructure requires defense in depth:

API Protection



Implement rate limiting & abuse detection at API edge to prevent DoS attacks and prompt injection

- Token bucket algorithms for fair resource allocation
- Pattern detection for malicious prompts

Data Security



Encrypt prompts/responses in transit and at rest to protect sensitive data and PII

- End-to-end encryption for confidential workloads
- Automatic PII detection and redaction

Compliance



Maintain comprehensive audit logs for regulatory compliance (HIPAA, GDPR, SOC2)

- Immutable logging of all model interactions
- Data lineage tracking for outputs

Access Control



Implement role-based access for multi-team use with granular permissions

- Model-level access controls
- Feature flagging for gradual rollout

Key Takeaways

Serving LLMs requires carefully balancing **latency, cost, and reliability** trade-offs

Critical optimizations include caching, batching, quantization, and intelligent routing

Infrastructure choices (Kubernetes, Ray, serverless) determine your scaling model and operational overhead

Governance, security, and comprehensive monitoring are **non-negotiable** in production environments

Next steps: Evaluate your current architecture against these patterns and identify optimization opportunities



📄 Additional Resources:

- vLLM documentation
- Ray Serve deployment guide
- NVIDIA TensorRT-LLM samples